Problem 1: Constructor Chaining

```
class A {
   A() {
      System.out.println("A");
   }
}

class B extends A {
   B() {
      System.out.println("B");
   }
}

class C extends B {
   C() {
      System.out.println("C");
   }

   public static void main(String[] args) {
      new C();
   }
}
```

Output

```
A
B
```

C

Problem 2: Constructor with Parameters + super()

```
class A {
  A(int x) {
    System.out.println(x);
  }
}

class B extends A {
  B() {
    super(10);
    System.out.println("B");
  }

  public static void main(String[] args) {
    new B();
  }
}
```

Problem 3: this() vs super() (Interview Trap)

```
class A {
  A() {
    System.out.println("A");
  }
}

class B extends A {
```

```
    B() {
        // this();     not allowed with super
        System.out.println("B");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

super() is implicitly called first

Problem 4: Method Overriding + Runtime Binding

```
class Parent {
    void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child");
    }

    public static void main(String[] args) {
        Parent p = new Child();
        p.show();
```

```
    }
}
```

Concept: Runtime polymorphism

Problem 5: Overriding + super.method()

```
class Parent {
    void work() {
        System.out.println("Parent Work");
    }
}

class Child extends Parent {
    void work() {
        super.work();
        System.out.println("Child Work");
    }

    public static void main(String[] args) {
        new Child().work();
    }
}
```

Problem 6: Overriding Rules – Access Modifier

```
class A {
    protected void show() {
        System.out.println("A");
```

```
    }
}

class B extends A {
    public void show() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Access can be broadened, not reduced

Problem 7: Overriding vs static Method (Hiding)

```
class A {
    static void show() {
        System.out.println("A");
    }
}

class B extends A {
    static void show() {
        System.out.println("B");
    }
```

```
    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Output

A

Problem 8: Instance Variable Hiding

```
class A {
    int x = 10;
}

class B extends A {
    int x = 20;

    void print() {
        System.out.println(x);
        System.out.println(super.x);
    }

    public static void main(String[] args) {
        new B().print();
    }
```

```
}
```

Problem 9: Polymorphism Limitation (Variables)

```
class A {
  int x = 10;
}

class B extends A {
  int x = 20;

  public static void main(String[] args) {
    A a = new B();
    System.out.println(a.x);
  }
}
```

Output

10

Variables are not polymorphic

Problem 10: Method Overloading vs Overriding

```
class A {
  void show(int x) {
    System.out.println("A int");
```

```
    }
}

class B extends A {
    void show(double x) {
        System.out.println("B double");
    }

    public static void main(String[] args) {
        B b = new B();
        b.show(10);
    }
}
```

Output

A int

Problem 11: Abstract Class – Partial Implementation

```
abstract class Vehicle {
    abstract void run();

    void start() {
        System.out.println("Vehicle Started");
    }
}
```

```
class Bike extends Vehicle {
   void run() {
      System.out.println("Bike Running");
   }

   public static void main(String[] args) {
      Vehicle v = new Bike();
      v.start();
      v.run();
   }
}
```

Problem 12: Abstract Reference with Multiple Children

```
abstract class Shape {
   abstract void draw();
}

class Circle extends Shape {
   void draw() {
      System.out.println("Circle");
   }
}

class Square extends Shape {
   void draw() {
      System.out.println("Square");
   }
```

```
    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Square();

        s1.draw();
        s2.draw();
    }
}
```

Problem 13: Interface vs Abstract (Constructor Difference)

```
interface A {
    void show();
}

abstract class B {
    B() {
        System.out.println("Constructor");
    }
}
```

Interfaces cannot have constructors

Problem 14: Interface Reference Polymorphism

```
interface Payment {
    void pay();
}
```

```
class CardPayment implements Payment {
   public void pay() {
      System.out.println("Card Payment");
   }
}

class UpiPayment implements Payment {
   public void pay() {
      System.out.println("UPI Payment");
   }
}

class Test {
   public static void main(String[] args) {
      Payment p1 = new CardPayment();
      Payment p2 = new UpiPayment();

      p1.pay();
      p2.pay();
   }
}
```

Problem 15: Multiple Interfaces – Real Use Case

```
interface Printable {
   void print();
}

interface Scannable {
```

```
    void scan();
}

class Printer implements Printable, Scannable {
    public void print() {
        System.out.println("Printing");
    }

    public void scan() {
        System.out.println("Scanning");
    }

    public static void main(String[] args) {
        Printer p = new Printer();
        p.print();
        p.scan();
    }
}
```

Problem 16: Tight Coupling (Bad Design)

```
class Engine {
    void start() {
        System.out.println("Engine Started");
    }
}

class Car {
    Engine e = new Engine();
```

```
    void drive() {
        e.start();
    }

    public static void main(String[] args) {
        new Car().drive();
    }
}
```

Problem 17: Loose Coupling (Better Design)

```
interface Engine {
    void start();
}

class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol Engine");
    }
}

class Car {
    Engine e;

    Car(Engine e) {
        this.e = e;
    }
```

```
    void drive() {
        e.start();
    }

    public static void main(String[] args) {
        Car c = new Car(new PetrolEngine());
        c.drive();
    }
}
```

Problem 18: Object as Method Argument (Polymorphism)

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

class Test {
    static void playSound(Animal a) {
        a.sound();
    }
```

```
    public static void main(String[] args) {
        playSound(new Dog());
    }
}
```

Problem 19: Method Returning Parent Type

```
class Animal {
    void type() {
        System.out.println("Animal");
    }
}

class Cat extends Animal {}

class Test {
    static Animal getAnimal() {
        return new Cat();
    }

    public static void main(String[] args) {
        Animal a = getAnimal();
        a.type();
    }
}
```

Problem 20: Constructor Injection (Interview Favorite)

```
interface Service {
    void execute();
```

```
}

class ServiceImpl implements Service {
   public void execute() {
      System.out.println("Service Executed");
   }
}

class Client {
   Service service;

   Client(Service s) {
      service = s;
   }

   void process() {
      service.execute();
   }

   public static void main(String[] args) {
      Client c = new Client(new ServiceImpl());
      c.process();
   }
}
```

Problem 21: Final Method – Cannot Override

```
class A {
   final void show() {
```

```
        System.out.println("Final Method");
    }
}

class B extends A {
    public static void main(String[] args) {
        new B().show();
    }
}
```

Problem 22: Final Variable Behavior

```
class Test {
    final int x;

    Test() {
        x = 10;
    }

    public static void main(String[] args) {
        System.out.println(new Test().x);
    }
}
```

Problem 23: Abstract Class with State

```
abstract class Employee {
    int salary;

    Employee(int s) {
```

```
        salary = s;
    }

    abstract void work();
}

class Developer extends Employee {
    Developer(int s) {
        super(s);
    }

    void work() {
        System.out.println("Coding");
    }

    public static void main(String[] args) {
        Employee e = new Developer(50000);
        System.out.println(e.salary);
        e.work();
    }
}
```

Problem 24: Real-Time Hierarchy (Company)

```
class Employee {
    void role() {
        System.out.println("Employee");
    }
}
```

```
class Manager extends Employee {
   void role() {
      System.out.println("Manager");
   }

   public static void main(String[] args) {
      Employee e = new Manager();
      e.role();
   }
}
```

Problem 25: Level-2 Summary Polymorphism

```
class A {
   void show() {
      System.out.println("A");
   }
}

class B extends A {
   void show() {
      System.out.println("B");
   }

   public static void main(String[] args) {
      A a = new B();
      a.show();
   }
```

```
}
```

Problem 1: Constructor Chaining (3-Level Inheritance)

```
class A {
  A() {
    System.out.println("A");
  }
}

class B extends A {
  B() {
    System.out.println("B");
  }
}

class C extends B {
  C() {
    System.out.println("C");
  }

  public static void main(String[] args) {
    new C();
  }
}
```

Output

```
A
B
C
```

Problem 2: Constructor with Parameters + super()

```
class A {
  A(int x) {
    System.out.println(x);
  }
}

class B extends A {
  B() {
    super(10);
    System.out.println("B");
  }

  public static void main(String[] args) {
    new B();
  }
}
```

Problem 3: this() vs super() (Interview Trap)

```
class A {
  A() {
    System.out.println("A");
  }
}

class B extends A {
  B() {
    // this();     not allowed with super
```

```
        System.out.println("B");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

super() is **implicitly called first**

Problem 4: Method Overriding + Runtime Binding

```
class Parent {
    void show() {
        System.out.println("Parent");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child");
    }

    public static void main(String[] args) {
        Parent p = new Child();
        p.show();
    }
}
```

Concept: Runtime polymorphism

Problem 5: Overriding + super.method()

```
class Parent {
  void work() {
    System.out.println("Parent Work");
  }
}

class Child extends Parent {
  void work() {
    super.work();
    System.out.println("Child Work");
  }

  public static void main(String[] args) {
    new Child().work();
  }
}
```

Problem 6: Overriding Rules – Access Modifier

```
class A {
  protected void show() {
    System.out.println("A");
  }
}

class B extends A {
  public void show() {
    System.out.println("B");
```

```
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Access can be **broadened**, not reduced

Problem 7: Overriding vs static Method (Hiding)

```
class A {
    static void show() {
        System.out.println("A");
    }
}

class B extends A {
    static void show() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Output

A

Problem 8: Instance Variable Hiding

```
class A {
  int x = 10;
}

class B extends A {
  int x = 20;

  void print() {
    System.out.println(x);
    System.out.println(super.x);
  }

  public static void main(String[] args) {
    new B().print();
  }
}
```

Problem 9: Polymorphism Limitation (Variables)

```
class A {
  int x = 10;
}

class B extends A {
  int x = 20;

  public static void main(String[] args) {
```

```
    A a = new B();
    System.out.println(a.x);
  }
}
```

Output

10

Variables are **not polymorphic**

Problem 10: Method Overloading vs Overriding

```
class A {
  void show(int x) {
    System.out.println("A int");
  }
}

class B extends A {
  void show(double x) {
    System.out.println("B double");
  }

  public static void main(String[] args) {
    B b = new B();
    b.show(10);
  }
}
```

Output

A int

Problem 11: Abstract Class – Partial Implementation

```
abstract class Vehicle {
   abstract void run();

   void start() {
      System.out.println("Vehicle Started");
   }
}

class Bike extends Vehicle {
   void run() {
      System.out.println("Bike Running");
   }

   public static void main(String[] args) {
      Vehicle v = new Bike();
      v.start();
      v.run();
   }
}
```

Problem 12: Abstract Reference with Multiple Children

```
abstract class Shape {
   abstract void draw();
}

class Circle extends Shape {
   void draw() {
```

```
        System.out.println("Circle");
    }
}

class Square extends Shape {
    void draw() {
        System.out.println("Square");
    }

    public static void main(String[] args) {
        Shape s1 = new Circle();
        Shape s2 = new Square();

        s1.draw();
        s2.draw();
    }
}
```

Problem 13: Interface vs Abstract (Constructor Difference)

```
interface A {
    void show();
}

abstract class B {
    B() {
        System.out.println("Constructor");
    }
}
```

Interfaces **cannot have constructors**

Problem 14: Interface Reference Polymorphism

```
interface Payment {
  void pay();
}

class CardPayment implements Payment {
  public void pay() {
    System.out.println("Card Payment");
  }
}

class UpiPayment implements Payment {
  public void pay() {
    System.out.println("UPI Payment");
  }
}

class Test {
  public static void main(String[] args) {
    Payment p1 = new CardPayment();
    Payment p2 = new UpiPayment();

    p1.pay();
    p2.pay();
  }
}
```

Problem 15: Multiple Interfaces – Real Use Case

```
interface Printable {
  void print();
}

interface Scannable {
  void scan();
}

class Printer implements Printable, Scannable {
  public void print() {
    System.out.println("Printing");
  }

  public void scan() {
    System.out.println("Scanning");
  }

  public static void main(String[] args) {
    Printer p = new Printer();
    p.print();
    p.scan();
  }
}
```

Problem 16: Tight Coupling (Bad Design)

```
class Engine {
```

```
    void start() {
        System.out.println("Engine Started");
    }
}

class Car {
    Engine e = new Engine();

    void drive() {
        e.start();
    }

    public static void main(String[] args) {
        new Car().drive();
    }
}
```

Problem 17: Loose Coupling (Better Design)

```
interface Engine {
    void start();
}

class PetrolEngine implements Engine {
    public void start() {
        System.out.println("Petrol Engine");
    }
}
```

```
class Car {
  Engine e;

  Car(Engine e) {
    this.e = e;
  }

  void drive() {
    e.start();
  }

  public static void main(String[] args) {
    Car c = new Car(new PetrolEngine());
    c.drive();
  }
}
```

Problem 18: Object as Method Argument (Polymorphism)

```
class Animal {
  void sound() {
    System.out.println("Animal Sound");
  }
}

class Dog extends Animal {
  void sound() {
    System.out.println("Bark");
  }
```

```
}

class Test {
   static void playSound(Animal a) {
      a.sound();
   }

   public static void main(String[] args) {
      playSound(new Dog());
   }
}
```

Problem 19: Method Returning Parent Type

```
class Animal {
   void type() {
      System.out.println("Animal");
   }
}

class Cat extends Animal {}

class Test {
   static Animal getAnimal() {
      return new Cat();
   }

   public static void main(String[] args) {
      Animal a = getAnimal();
```

```
        a.type();
    }
}
```

Problem 20: Constructor Injection (Interview Favorite)

```
interface Service {
    void execute();
}

class ServiceImpl implements Service {
    public void execute() {
        System.out.println("Service Executed");
    }
}

class Client {
    Service service;

    Client(Service s) {
        service = s;
    }

    void process() {
        service.execute();
    }

    public static void main(String[] args) {
        Client c = new Client(new ServiceImpl());
```

```
        c.process();
    }
}
```

Problem 21: Final Method – Cannot Override

```
class A {
    final void show() {
        System.out.println("Final Method");
    }
}

class B extends A {
    public static void main(String[] args) {
        new B().show();
    }
}
```

Problem 22: Final Variable Behavior

```
class Test {
    final int x;

    Test() {
        x = 10;
    }

    public static void main(String[] args) {
        System.out.println(new Test().x);
    }
```

```
}
```

Problem 23: Abstract Class with State

```
abstract class Employee {
  int salary;

  Employee(int s) {
    salary = s;
  }

  abstract void work();
}

class Developer extends Employee {
  Developer(int s) {
    super(s);
  }

  void work() {
    System.out.println("Coding");
  }

  public static void main(String[] args) {
    Employee e = new Developer(50000);
    System.out.println(e.salary);
    e.work();
  }
}
```

Problem 24: Real-Time Hierarchy (Company)

```
class Employee {
  void role() {
    System.out.println("Employee");
  }
}

class Manager extends Employee {
  void role() {
    System.out.println("Manager");
  }

  public static void main(String[] args) {
    Employee e = new Manager();
    e.role();
  }
}
```

Problem 25: Level-2 Summary Polymorphism

```
class A {
  void show() {
    System.out.println("A");
  }
}

class B extends A {
  void show() {
```

```
        System.out.println("B");
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Problem 26: Constructor Overloading + Inheritance

```
class A {
    A() {
        System.out.println("A Default");
    }

    A(int x) {
        System.out.println("A Param " + x);
    }
}

class B extends A {
    B() {
        super(10);
        System.out.println("B Default");
    }

    public static void main(String[] args) {
        new B();
```

```
    }
}
```

Problem 27: Implicit super() Call

```
class A {
    A() {
        System.out.println("A Constructor");
    }
}

class B extends A {
    B() {
        System.out.println("B Constructor");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

Problem 28: Method Overriding + Upcasting

```
class A {
    void display() {
        System.out.println("A");
    }
}

class B extends A {
```

```
    void display() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        A obj = new B();
        obj.display();
    }
}
```

Problem 29: Overloading Not Polymorphism

```
class Test {
    void show(int x) {
        System.out.println("int");
    }

    void show(double x) {
        System.out.println("double");
    }

    public static void main(String[] args) {
        Test t = new Test();
        t.show(10);
    }
}
```

Problem 30: Overriding + Reference Type Check

```
class A {
```

```
  void show() {
    System.out.println("A");
  }
}

class B extends A {
  void show() {
    System.out.println("B");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

Problem 31: super Keyword – Variable Access

```
class A {
  int x = 100;
}

class B extends A {
  int x = 200;

  void print() {
    System.out.println(x);
    System.out.println(super.x);
  }
```

```
    public static void main(String[] args) {
        new B().print();
    }
}
```

Problem 32: Static Variable in Inheritance

```
class A {
    static int x = 10;
}

class B extends A {
    static int x = 20;

    public static void main(String[] args) {
        System.out.println(A.x);
        System.out.println(B.x);
    }
}
```

Problem 33: Static Method Hiding

```
class A {
    static void show() {
        System.out.println("A");
    }
}

class B extends A {
```

```
    static void show() {
        System.out.println("B");
    }

    public static void main(String[] args) {
        A a = new B();
        a.show();
    }
}
```

Output: A

Problem 34: Abstract Class with State + Behavior

```
abstract class Account {
    int balance;

    Account(int b) {
        balance = b;
    }

    abstract void calculateInterest();
}

class SavingsAccount extends Account {
    SavingsAccount(int b) {
        super(b);
    }

    void calculateInterest() {
```

```
    System.out.println(balance * 0.05);
  }

  public static void main(String[] args) {
    Account a = new SavingsAccount(10000);
    a.calculateInterest();
  }
}
```

Problem 35: Abstract Method Called via Constructor

```
abstract class A {
  A() {
    show();
  }

  abstract void show();
}

class B extends A {
  int x = 10;

  void show() {
    System.out.println(x);
  }

  public static void main(String[] args) {
    new B();
  }
```

```
}
```

⚠️ **Interview trap**: subclass fields not initialized yet

Problem 36: Interface Variable Sharing

```
interface Config {
  int TIMEOUT = 30;
}

class App {
  public static void main(String[] args) {
    System.out.println(Config.TIMEOUT);
  }
}
```

Problem 37: Interface Method Polymorphism

```
interface Notification {
  void send();
}

class SMS implements Notification {
  public void send() {
    System.out.println("SMS Sent");
  }
}

class Email implements Notification {
  public void send() {
    System.out.println("Email Sent");
```

```
    }
}

class Test {
    public static void main(String[] args) {
        Notification n = new Email();
        n.send();
    }
}
```

Problem 38: Multiple Interface References

```
interface A {
    void a();
}

interface B {
    void b();
}

class C implements A, B {
    public void a() {
        System.out.println("A");
    }

    public void b() {
        System.out.println("B");
    }
```

```
    public static void main(String[] args) {
        A obj1 = new C();
        B obj2 = new C();

        obj1.a();
        obj2.b();
    }
}
```

Problem 39: Loose Coupling via Method Parameter

```
interface Payment {
    void pay();
}

class Cash implements Payment {
    public void pay() {
        System.out.println("Cash Payment");
    }
}

class Shop {
    void process(Payment p) {
        p.pay();
    }

    public static void main(String[] args) {
        Shop s = new Shop();
        s.process(new Cash());
```

```
    }
}
```

Problem 40: Method Returning Interface

```
interface Shape {
    void draw();
}

class Circle implements Shape {
    public void draw() {
        System.out.println("Circle");
    }
}

class Factory {
    static Shape getShape() {
        return new Circle();
    }

    public static void main(String[] args) {
        Shape s = Factory.getShape();
        s.draw();
    }
}
```

Problem 41: Abstract Class Reference vs Object

```
abstract class A {
    abstract void show();
```

```
}

class B extends A {
  void show() {
    System.out.println("Hello");
  }

  public static void main(String[] args) {
    A a = new B();
    a.show();
  }
}
```

Problem 42: Encapsulation + Business Rule

```
class Product {
  private int price;

  void setPrice(int p) {
    if (p > 0)
      price = p;
  }

  int getPrice() {
    return price;
  }

  public static void main(String[] args) {
    Product p = new Product();
```

```
        p.setPrice(500);
        System.out.println(p.getPrice());
    }
}
```

Problem 43: Constructor Injection (Real-Time)

```
interface Logger {
    void log();
}

class FileLogger implements Logger {
    public void log() {
        System.out.println("File Log");
    }
}

class Service {
    Logger logger;

    Service(Logger l) {
        logger = l;
    }

    void execute() {
        logger.log();
    }

    public static void main(String[] args) {
```

```
        Service s = new Service(new FileLogger());
        s.execute();
    }
}
```

Problem 44: Polymorphic Array (OOPS Only)

```
class Animal {
    void sound() {
        System.out.println("Animal");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Bark");
    }
}

class Test {
    public static void main(String[] args) {
        Animal[] animals = { new Dog(), new Dog() };

        for (Animal a : animals) {
            a.sound();
        }
    }
}
```

Problem 45: instanceof with Polymorphism

```
class A {}
class B extends A {}

class Test {
  public static void main(String[] args) {
    A a = new B();
    System.out.println(a instanceof B);
    System.out.println(a instanceof A);
  }
}
```

Problem 46: Final Variable in Constructor

```
class Test {
  final int x;

  Test(int a) {
    x = a;
  }

  public static void main(String[] args) {
    System.out.println(new Test(10).x);
  }
}
```

Problem 47: Abstract Class with Multiple Methods

```
abstract class Employee {
  abstract void work();
```

```
    void login() {
        System.out.println("Login");
    }
}

class Developer extends Employee {
    void work() {
        System.out.println("Coding");
    }

    public static void main(String[] args) {
        Employee e = new Developer();
        e.login();
        e.work();
    }
}
```

Problem 48: Overloading vs Overriding Confusion

```
class A {
    void show(int x) {
        System.out.println("A int");
    }
}

class B extends A {
    void show(int x) {
        System.out.println("B int");
```

```
    }

    public static void main(String[] args) {
        A a = new B();
        a.show(10);
    }
}
```

Problem 49: Service Layer Pattern (Simple)

```
class OrderService {
    void placeOrder() {
        System.out.println("Order Placed");
    }

    public static void main(String[] args) {
        new OrderService().placeOrder();
    }
}
```

Problem 50: Level-2 Mid Summary Example

```
interface Operation {
    void execute();
}

class AddOperation implements Operation {
    public void execute() {
        System.out.println("Add Operation");
    }
```

```
}

class Client {
  public static void main(String[] args) {
    Operation op = new AddOperation();
    op.execute();
  }
}
```

LEVEL 2 – PART 3 (Problems 51–75)

Problem 51: Constructor + Method Overriding Order

```
class A {
  A() {
    show();
  }

  void show() {
    System.out.println("A");
  }
}

class B extends A {
  int x = 10;

  void show() {
    System.out.println(x);
  }
```

```
    public static void main(String[] args) {
        new B();
    }
}
```

Output: 0 (instance variable not initialized yet)

Problem 52: Overriding Called via super()

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        System.out.println("B");
    }

    void call() {
        super.show();
        show();
    }

    public static void main(String[] args) {
        new B().call();
    }
}
```

Problem 53: Constructor Injection + Polymorphism

```
interface DB {
  void connect();
}

class MySQL implements DB {
  public void connect() {
    System.out.println("MySQL Connected");
  }
}

class App {
  DB db;

  App(DB d) {
    db = d;
  }

  void start() {
    db.connect();
  }

  public static void main(String[] args) {
    new App(new MySQL()).start();
  }
}
```

Problem 54: Method Argument as Parent Type

```
class Shape {
  void draw() {
    System.out.println("Shape");
  }
}

class Circle extends Shape {
  void draw() {
    System.out.println("Circle");
  }
}

class Painter {
  void paint(Shape s) {
    s.draw();
  }

  public static void main(String[] args) {
    new Painter().paint(new Circle());
  }
}
```

Problem 55: Method Returning Parent Reference

```
class Animal {
  void sound() {
    System.out.println("Animal");
  }
```

```
}

class Dog extends Animal {
  void sound() {
    System.out.println("Bark");
  }
}

class Factory {
  static Animal get() {
    return new Dog();
  }

  public static void main(String[] args) {
    Animal a = get();
    a.sound();
  }
}
```

Problem 56: Variable Shadowing vs Polymorphism

```
class A {
  int x = 10;
}

class B extends A {
  int x = 20;
}
```

```
class Test {
   public static void main(String[] args) {
      A a = new B();
      System.out.println(a.x);
   }
}
```

Output: 10

Problem 57: Static Variable Shared in Inheritance

```
class A {
   static int x = 10;
}

class B extends A {
   static int x = 20;
}

class Test {
   public static void main(String[] args) {
      System.out.println(A.x);
      System.out.println(B.x);
   }
}
```

Problem 58: Static Method Hiding – Reference Matters

```
class A {
   static void show() {
      System.out.println("A");
```

```
    }
}

class B extends A {
    static void show() {
        System.out.println("B");
    }
}

class Test {
    public static void main(String[] args) {
        A a = new B();
        B b = new B();

        a.show();
        b.show();
    }
}
```

Problem 59: Abstract Class + Constructor Execution

```
abstract class A {
    A() {
        System.out.println("A Constructor");
    }
}

class B extends A {
    B() {
```

```
        System.out.println("B Constructor");
    }

    public static void main(String[] args) {
        new B();
    }
}
```

Problem 60: Abstract Method Called via Reference

```
abstract class Machine {
    abstract void start();
}

class Printer extends Machine {
    void start() {
        System.out.println("Printer Started");
    }

    public static void main(String[] args) {
        Machine m = new Printer();
        m.start();
    }
}
```

Problem 61: Interface Default Access Rules

```
interface A {
    void show();
}
```

```
class B implements A {
   public void show() {
      System.out.println("Implemented");
   }

   public static void main(String[] args) {
      A a = new B();
      a.show();
   }
}
```

Problem 62: Interface Reference – Loose Coupling

```
interface Storage {
   void save();
}

class FileStorage implements Storage {
   public void save() {
      System.out.println("Saved in File");
   }
}

class App {
   Storage storage;

   App(Storage s) {
      storage = s;
```

```
    }

    void process() {
        storage.save();
    }

    public static void main(String[] args) {
        new App(new FileStorage()).process();
    }
}
```

Problem 63: Multiple Interface Polymorphism

```
interface Readable {
    void read();
}

interface Writable {
    void write();
}

class File implements Readable, Writable {
    public void read() {
        System.out.println("Reading");
    }

    public void write() {
        System.out.println("Writing");
    }
```

```
    public static void main(String[] args) {
        Readable r = new File();
        r.read();

        Writable w = new File();
        w.write();
    }
}
```

Problem 64: instanceof with Interface

```
interface A {}
class B implements A {}

class Test {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a instanceof A);
        System.out.println(a instanceof B);
    }
}
```

Problem 65: Abstract Class with Concrete Method Usage

```
abstract class Report {
    void header() {
        System.out.println("Header");
    }
```

```
    abstract void body();
}

class SalesReport extends Report {
    void body() {
        System.out.println("Sales Data");
    }

    public static void main(String[] args) {
        Report r = new SalesReport();
        r.header();
        r.body();
    }
}
```

Problem 66: Constructor Overloading in Child

```
class A {
    A(int x) {
        System.out.println(x);
    }
}

class B extends A {
    B() {
        super(100);
    }

    B(int y) {
```

```
        super(y);
    }

    public static void main(String[] args) {
        new B();
        new B(200);
    }
}
```

Problem 67: this vs super in Methods

```
class A {
    void show() {
        System.out.println("A");
    }
}

class B extends A {
    void show() {
        System.out.println("B");
    }

    void test() {
        this.show();
        super.show();
    }

    public static void main(String[] args) {
        new B().test();
```

```
    }
}
```

Problem 68: Method Overriding + Return Type (Covariant)

```
class A {
    Object get() {
        return "Hello";
    }
}

class B extends A {
    String get() {
        return "World";
    }

    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.get());
    }
}
```

Problem 69: Abstract Class with State + Polymorphism

```
abstract class Account {
    int balance;

    Account(int b) {
        balance = b;
    }
```

```
  abstract void calculate();
}

class LoanAccount extends Account {
  LoanAccount(int b) {
    super(b);
  }

  void calculate() {
    System.out.println(balance * 0.1);
  }

  public static void main(String[] args) {
    Account a = new LoanAccount(10000);
    a.calculate();
  }
}
```

Problem 70: Object Dependency via Method Parameter

```
interface Printer {
  void print();
}

class HPPrinter implements Printer {
  public void print() {
    System.out.println("HP Printing");
  }
```

```
}

class Office {
   void work(Printer p) {
      p.print();
   }

   public static void main(String[] args) {
      new Office().work(new HPPrinter());
   }
}
```

Problem 71: Abstract Class Reference Array

```
abstract class Shape {
   abstract void draw();
}

class Circle extends Shape {
   void draw() {
      System.out.println("Circle");
   }
}

class Square extends Shape {
   void draw() {
      System.out.println("Square");
   }
}
```

```
class Test {
   public static void main(String[] args) {
      Shape[] shapes = { new Circle(), new Square() };

      for (Shape s : shapes) {
         s.draw();
      }
   }
}
```

Problem 72: Interface Reference Array

```
interface Operation {
   void execute();
}

class Add implements Operation {
   public void execute() {
      System.out.println("Add");
   }
}

class Sub implements Operation {
   public void execute() {
      System.out.println("Sub");
   }
}
```

```
class Test {
  public static void main(String[] args) {
    Operation[] ops = { new Add(), new Sub() };

    for (Operation o : ops) {
      o.execute();
    }
  }
}
```

Problem 73: Polymorphic Method Call Chain

```
class A {
  void show() {
    System.out.println("A");
  }
}

class B extends A {
  void show() {
    System.out.println("B");
  }
}

class C extends B {
  void show() {
    System.out.println("C");
  }
```

```
    public static void main(String[] args) {
        A a = new C();
        a.show();
    }
}
```

Problem 74: Final Method in Inheritance

```
class A {
    final void show() {
        System.out.println("Final Show");
    }
}

class B extends A {
    public static void main(String[] args) {
        new B().show();
    }
}
```

Problem 75: Level-2 Real-Time Mini Design

```
interface AuthService {
    void login();
}

class BasicAuth implements AuthService {
    public void login() {
        System.out.println("Login Successful");
    }
```

```
}

class Application {
  AuthService auth;

  Application(AuthService a) {
    auth = a;
  }

  void start() {
    auth.login();
  }

  public static void main(String[] args) {
    new Application(new BasicAuth()).start();
  }
}
```